

ICE-4101

Advance Artificial Intelligence



Lecture-8

Tree Search, Graph Search

Course Teacher

Prof. Dr. Md. Ashikur Rahman Khan

Searching for Solutions

Having formulated some problems, we now need to solve them.

A solution is an action sequence, so

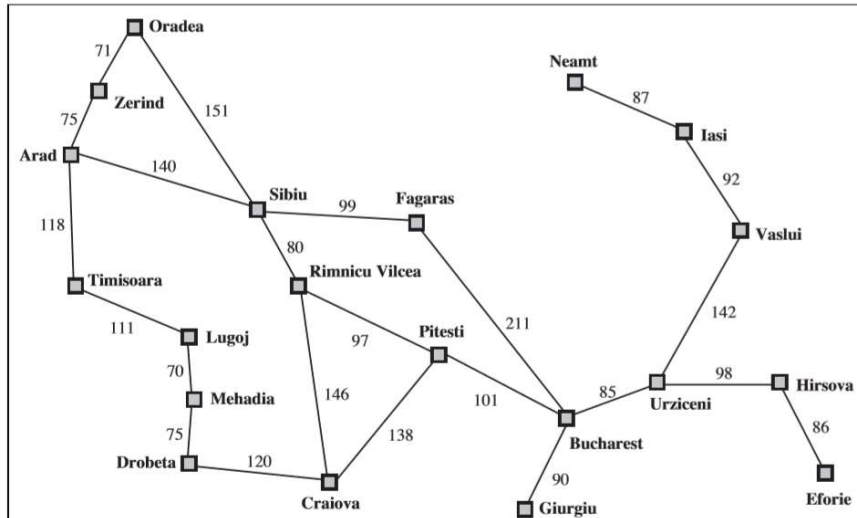
search algorithms work by considering various possible action sequences.

The possible action sequences starting at the initial state form a **search tree** with the initial state at the root; the branches are actions and the **nodes** correspond to states in the state space of the problem. **Or**

A search tree is generated by generating search nodes by successively expanding states starting from the initial state as the root.

Searching for Solutions

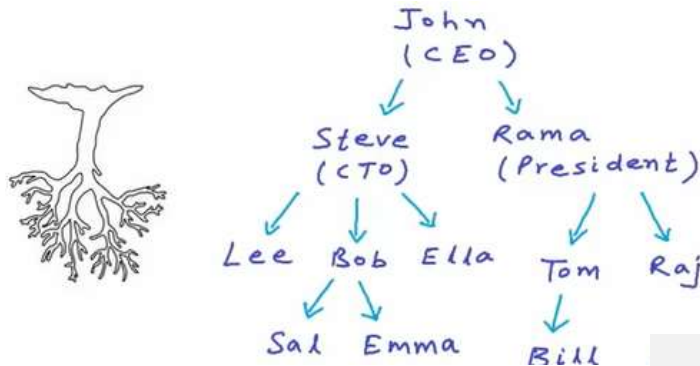
Following Figure shows the first few steps in growing the search tree for finding a route from Arad to Bucharest. The root node of the tree corresponds to the initial state, $In(Arad)$.



Tree

- i. Root
- ii. Children
- iii. Parent
- iv. Sibling
- v. Leaf
- vi. Cousin

Tree (Continued)

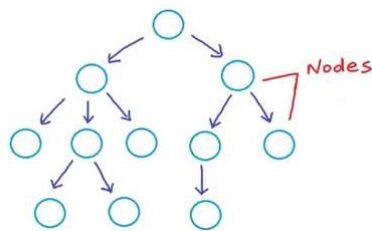


Here the data is **name of employee** and **designation**

So the object here:

One to store name and
another **to store designation**

Tree (Continued)

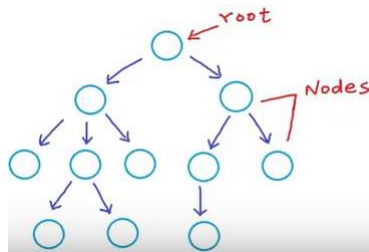


i. Root:

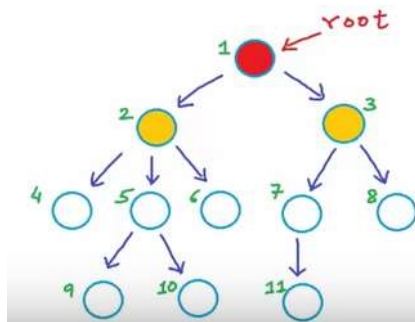
The **top most** node in the tree is called **root** of the tree.

Root is the only node without a parent.

Each node will **contain some data** and
this can be **data of any type**.



Tree (Continued)



ii. Children:

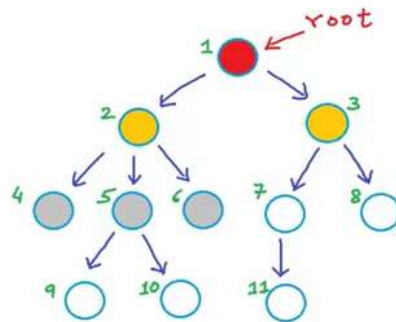
A node may have link or reference to some other nodes that will be called its children.

2 and 3 are children of 1.

iii. Parent:

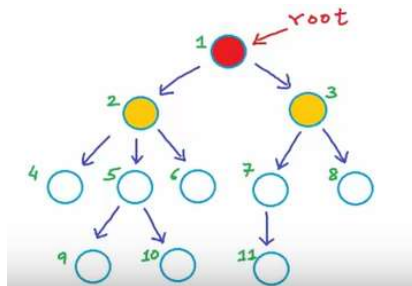
And node 1 is parent of nodes 2 and 3.

Tree (Continued)



So node 2 is child of node 1,
but parent of nodes 4, 5 and 6.

Tree (Continued)



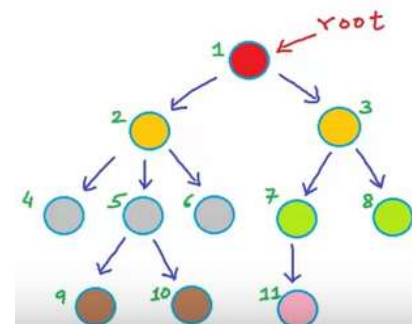
iv. Sibling:

Children of same parent are called **sibling**.

4, 5 and 6 are sibling,

7, 8 are sibling and finally

9 and 10 are sibling.



Tree (Continued)

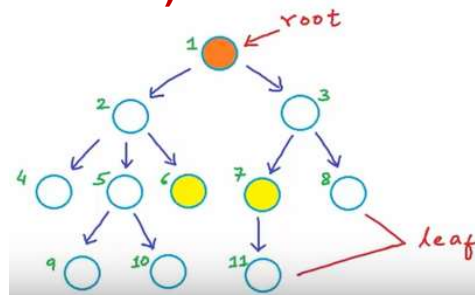
Are 6 and 7 sibling?

Sibling must have same parent.

Nodes not having same parent but having same grand parent can be called cousins.

So 6 and 7 are cousins.

We can say that node number 3 is the uncle of node number 6 because it's sibling of 2.



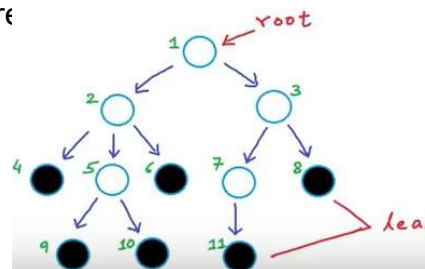
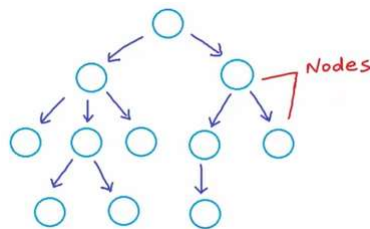
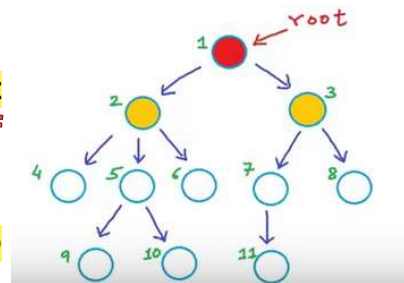
Tree (Continued)

v. Leaf:

Any node in the tree that does **not** have a child is called **leaf node**.

A leaf node is a node with zero child.

All nodes marked in black here are leaves.

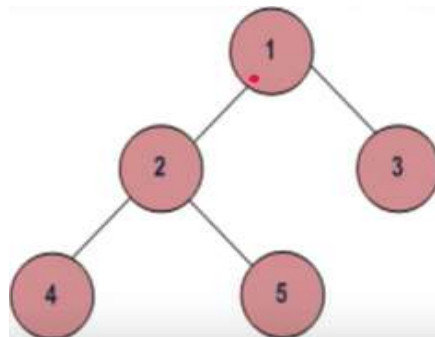


Leaf

Both left and right child nodes are Null.

Which one of the following figure is leaf node.

In the given binary tree we can, several nodes which can be classified as leaf nodes, we first see node 3. This node has both left and right child nodes null. Thus 3 is a leaf node.



Next both 4 & 5 have left and right child nodes null. These two nodes are leaf node.

Leaf

Next both 4 & 5 have left and right child nodes null. These two nodes are leaf node.

The node 2 cannot be a leaf node since it has both left and right child nodes.

And so finally, the leaf count for this given binary tree is 3.

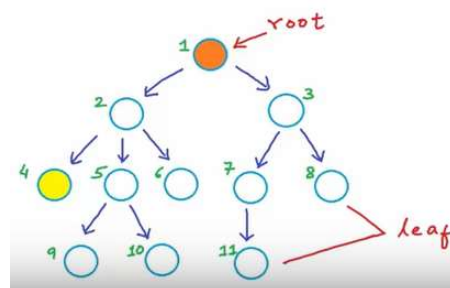
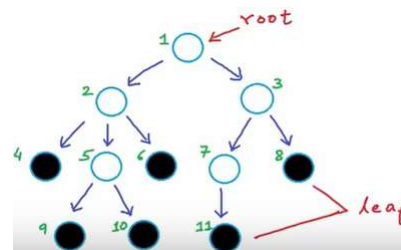
Tree (Continued)

Parent of parent can be called **grand-parent**.

So

1 is grand-parent of 4 and

4 is grand-child of 1.



Tree (Continued)

We have a link from 1 to 2,

so

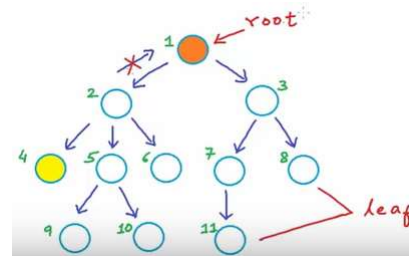
we can go from 1 to 2, but

we cannot go from 2 to 1.

If it is not bidirectional,

we can go from 1 to 2 but

we cannot go from 2 to 1.

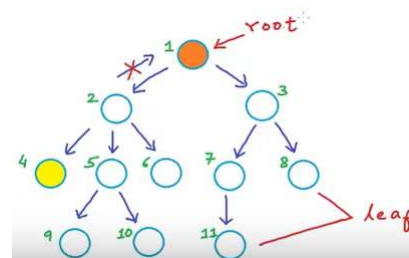


Tree (Continued)

i. Ancestor and

ii. Descendant:

When we are walking or travelling the tree, we can walk in only one direction.



So if we can go from node A to node B,

then A can be called ancestor of B and

B can be called descendant of A.

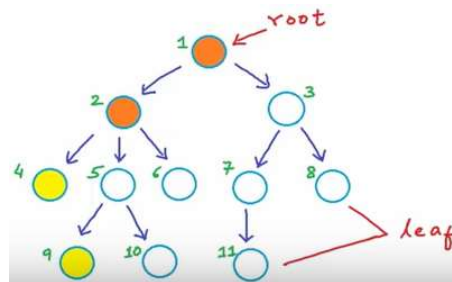
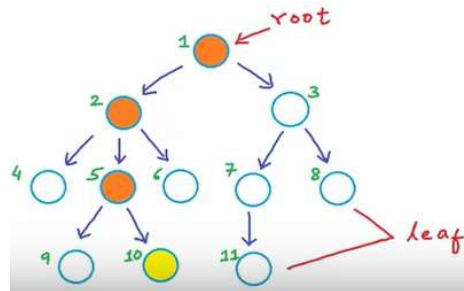
Tree (Continued)

For node 10

1, 2 and 5 are all ancestor of 10

and 10 is a descendant of all of these nodes.

We can walk from any of these nodes to 10.



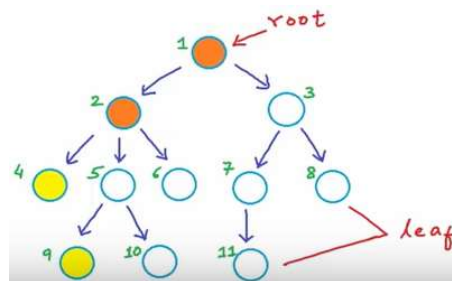
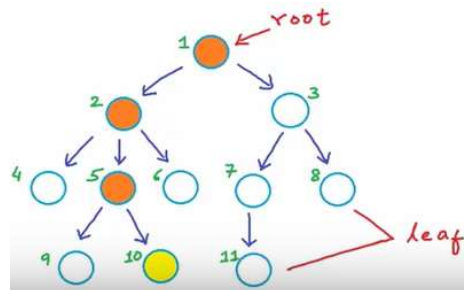
Tree (Continued)

What are the common ancestors of 4 and 9?

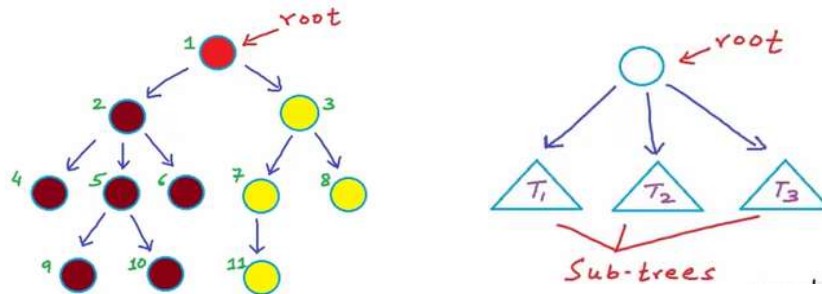
Ancestors of 4 are 1 and 2.

Ancestors of 9 are 1, 2 and 5.

So common ancestors will be 1 and 2.



Structure of Tree

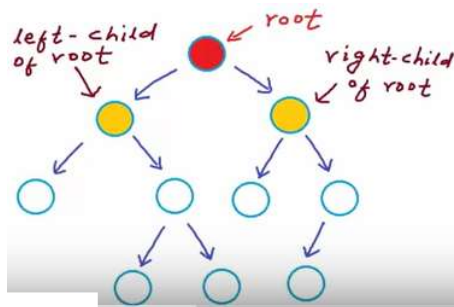


We can define tree recursively as a structure that consists of a distinguished node **called root** and some **sub-trees**.

The arrangement is such that root of the tree contains links to roots of all the sub-trees.

T_1 , T_2 and T_3 in this figure are sub-trees.

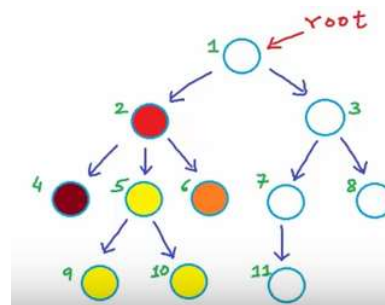
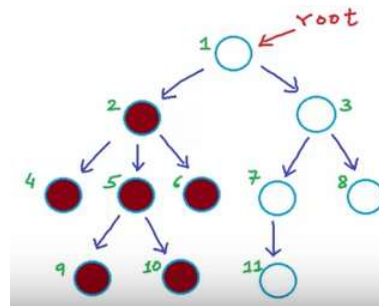
Structure of Tree



Sub Tree

We can further split the left sub-tree look at it like node number 2 is root of this sub-tree

This particular tree with node number 2 as root has three sub-trees shown in three different colors.

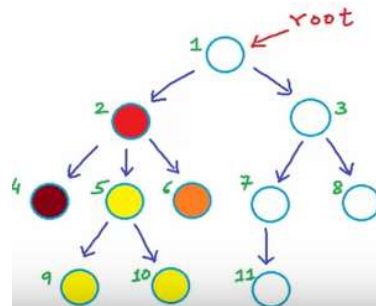


Sub Tree

In a tree with n nodes, there will be exactly $n-1$ links or edges.

Each arrow in this figure is a link or an edge.

All nodes except the root node will have exactly 1 incoming edge.



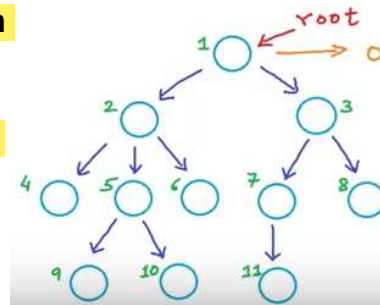
Depth of Node

Depth of some node X in a tree can be defined as length of the path from **root to node X**.

Each edge in the path will contribute one unit to the length.

We can say number of edges in path from root to X.

The depth of root node is zero.



Depth of Node

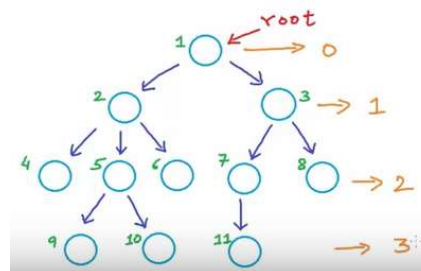
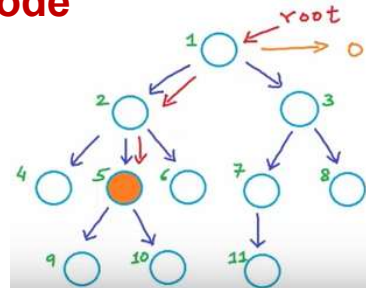
For node 5, we have 2 edges in the path from root

So the depth of the node, 5 is 2.

Depth of the node 2 and 3 is 1.

Depth of node 4, 5, 6, 7 and 8 is 2. and

Depth of the node, 9, 10, and 11 is 3.



Height of Node

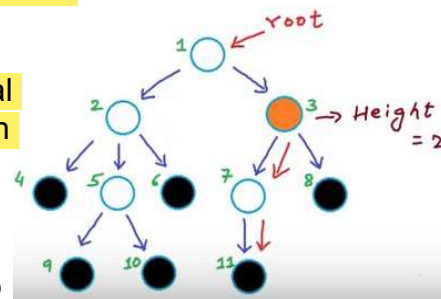
Height is defined as the number of edges in longest path **from that node to leaf node**.

So, height of some node X is equal to number of edges in longest path from X to a leaf.

Here, for node 3:

The longest path from this node to any leaf is 2.

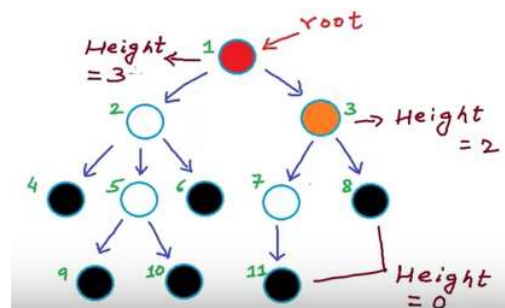
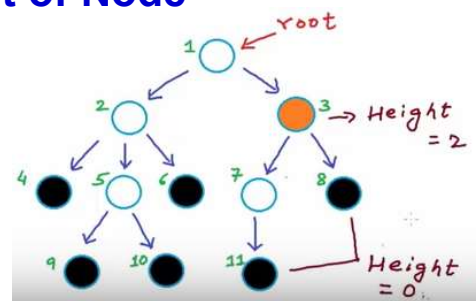
So height of node 3 is 2.



Height of Node

Height of leaf nodes is zero.

Height of the root node here is 3.

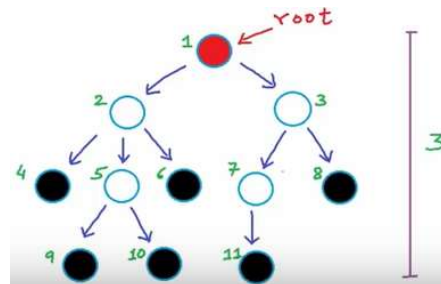


Height of Node

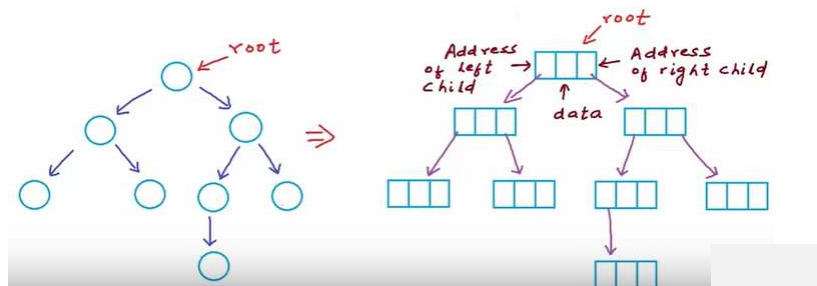
Height of tree is defined as height of root node

Height of this tree in figure is 3.

Height and depth of a tree may not may not be same.



Node?

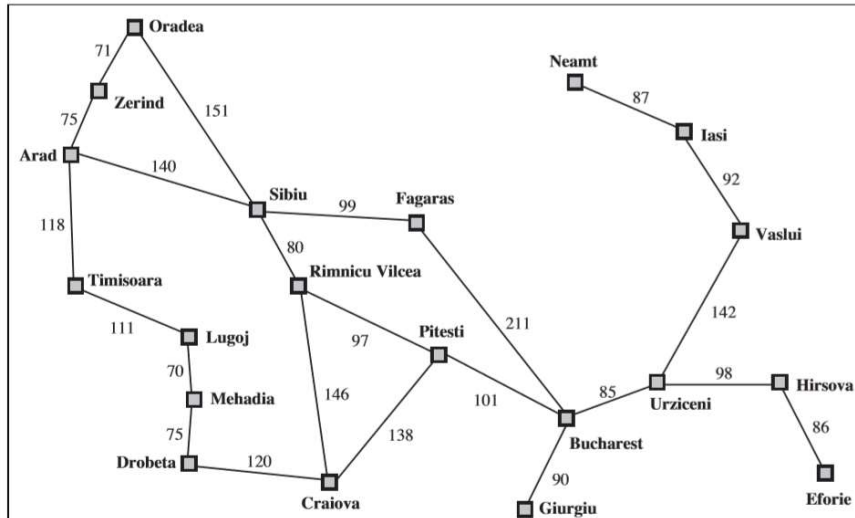


Node has three fields.

- One of the fields is to store data.
- The left cell is to store the address of the left child.
- And the right cell is to store address of the right child.

Searching for Solutions

Following Figure shows the first few steps in growing the search tree for finding a route from Arad to Bucharest. The root node of the tree corresponds to the initial state, $In(Arad)$.

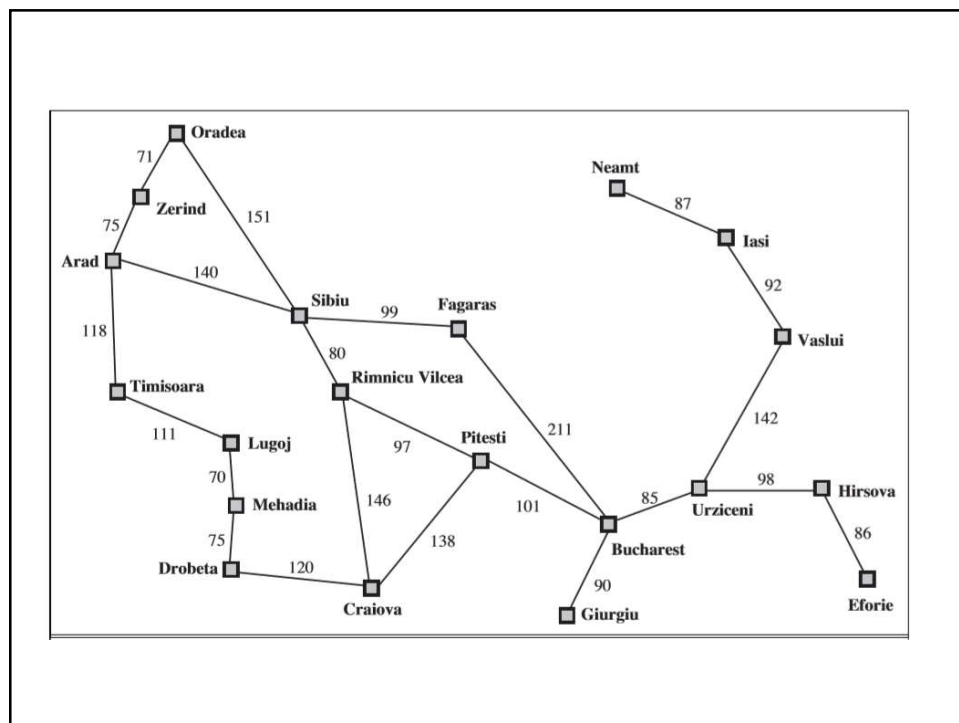
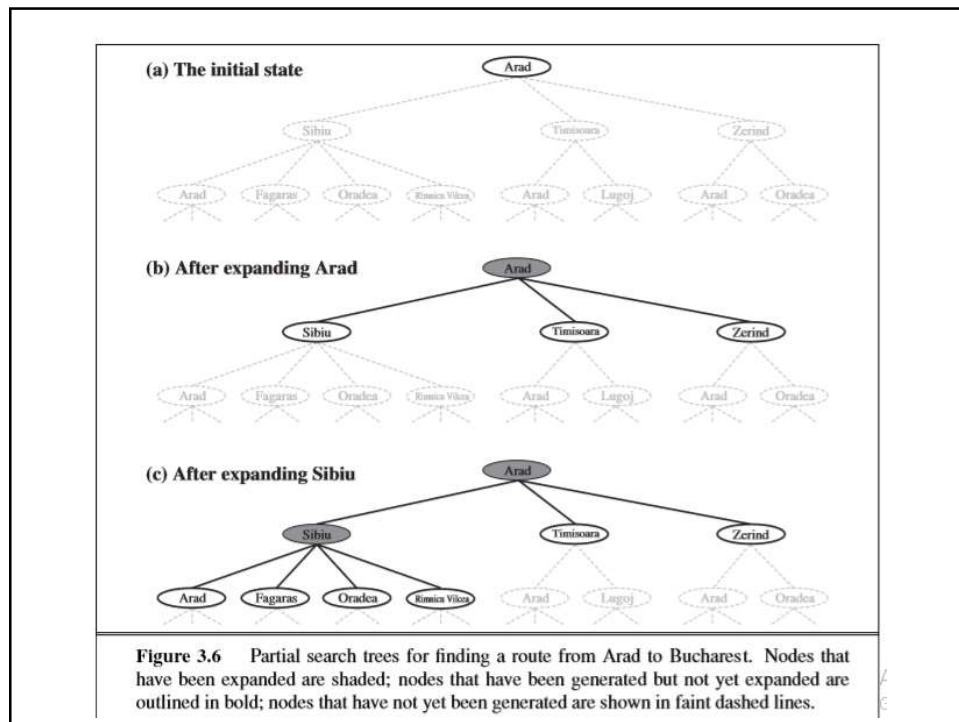


i. The first step is to test whether this is a goal state.

Clearly it is not, but it is important to check so that we can solve trick problems like “starting in Arad, get to Arad.”

ii. Then we need to consider taking various actions.

We do this by expanding the current state; that is applying each legal action to the current state, thereby generating a new set of states.



Parent node, Child node and Leaf node

In this case, we add three branches from the **parent node** In(Arad) leading to three new **child nodes**: In(Sibiu), In(Timisoara), and In(Zerind).

Now we must choose which of these three possibilities to consider further.

This is the essence of search—following up one option now and

putting the others aside for later, in case the first choice does not lead to a solution.

Parent node, Child node and Leaf node

Suppose we choose **Sibiu** first.

We check to see **whether it is a goal state** (it is not)

and **then expand** it to get In(Arad), In(Fagaras), In(Oradea), and In(Rimnicu Vilcea).

We can then choose any of these four or go back and choose Timisoara or Zerind.

Each of these six nodes is a leaf node, that is, a node with no children in the tree.

Parent node, Child node and Leaf node

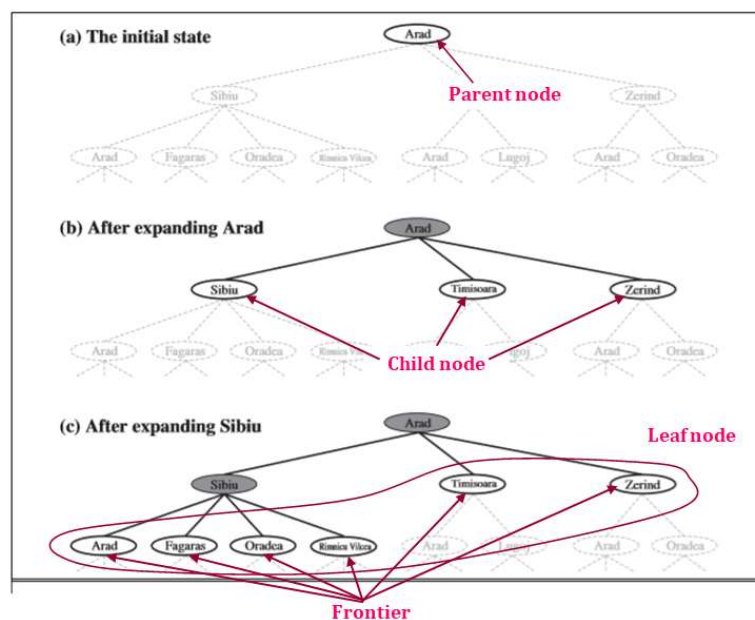
The set of all leaf nodes available for expansion at any given point is called the **frontier**.

(Many authors call it the open list, which is both geographically less evocative and less accurate, because other data structures are better suited than a list.)

In Figure 3.6, the **frontier** of each tree consists of those nodes with bold outlines.

The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.

Parent node, Child node and Leaf node



Classification of Tree

Trees are classified into various categories

There are different kinds of trees that are used in different scenarios.

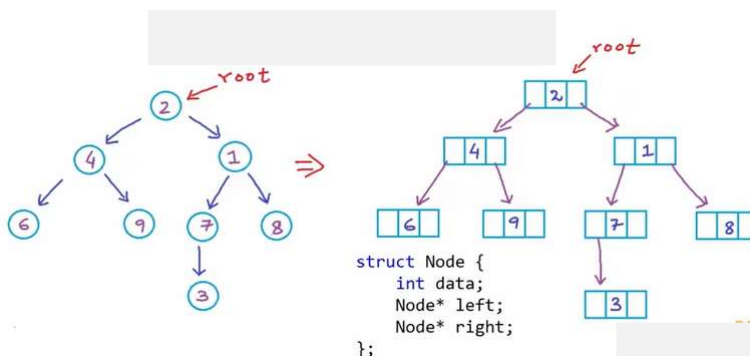
- ✓ **Binary Tree**
- ✓ **Generic Tree**

Binary tree is simplest and most common and famous tree.

Binary tree is tree in which each node can have at most 2 children (i.e not more than two children).

Binary Tree

An integer variable to store the data and then we have to pointers to node, one to store the address of the left child that will be the root of the left sub-tree and another to store the address of the right child.



Binary Tree

Generic Tree

Another one is generic tree that can have any number of children.

Applications:

- 1) Storing naturally hierarchical data → eg:- file system
- 2) Organize data for quick search, insertion, deletion → eg:- Binary Search trees
- 3) Trie → dictionary
- 4) Network Routing algorithm

A Generic Searching

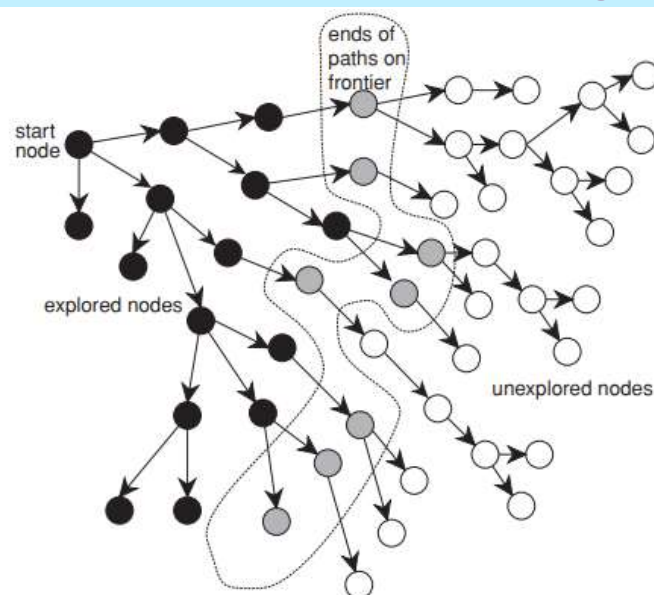
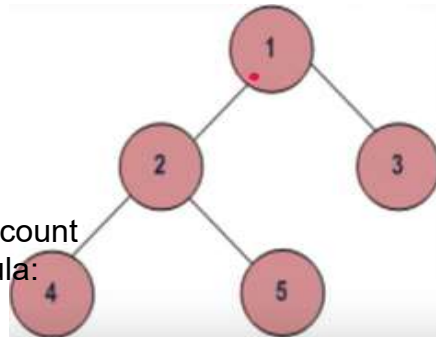


Figure 3.3: Problem solving by graph searching

Algorithm to calculate the number Leaf Node in a binary tree

The algorithm to calculate the number Leaf Node in a binary tree.

1. If node is NULL
Return 0
2. Else if left and right child nodes are Null return 1
3. Else recursively calculate leaf count of the binary tree using the formula:



Leaf count of a tree = Leaf count of the left sub tree + leaf count of right sub tree

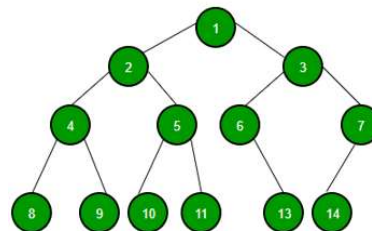
Tree-Search Algorithm

Tree:

A tree is a finite set of one or more nodes such that –

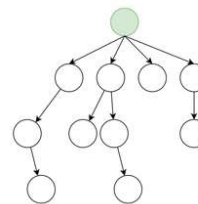
There is a specially designated node called root.

The concept of tree is represented by following Fig.



The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, T_2, T_3, \dots, T_n$

where $T_1, T_2, T_3, \dots, T_n$ is called the subtrees of the root.



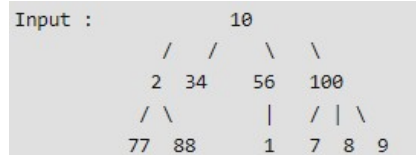
General Tree

General Tree-Search Algorithm

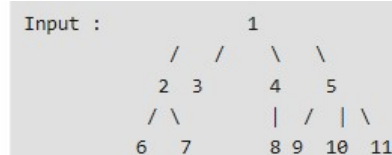
General Tree (Each node can have arbitrary number of children) Level Order Traversal

Given a generic tree, perform a Level order traversal and print all of its nodes

Examples:



Output : 10
2 34 56 100
77 88 1 7 8 9



Output : 1
2 3 4 5
6 7 8 9 10 11

Tree-Search Algorithm

Tree-search algorithm is the process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.

The general TREE-SEARCH algorithm is shown in Figure 3.7 (i).

Search algorithm is the easiest way to implement various search strategies is to maintain a queue of unexpanded search nodes.

General Tree-Search Algorithm

Search strategy:

The way how to choose which state to expand next is called search strategy.

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Figure 3.7 (i) A general tree-search algorithms

Tree Search

```
open <- [ ]
next <- start
while next is not goal {
  add all successors of
  next to open
  next <- select one node from open
  remove next from open
}
return next
```

General Tree-Search Algorithm

General Tree Search Algorithm

 General Tree Search Algorithm.py

```

1  def TreeSearch(problem, strategy):
2      Initialize search tree using the state of problem
3      while True:
4          if there are no candidates for expansion:
5              return failure
6          choose a leaf node for expansion according to strategy
7          if the node contains a goal state:
8              return the corresponding solution
9          else:
10             expand the node and add its successors to the search tree

```

Tree Search

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty
state, some description of the current world state
goal, a goal, initially null
problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

if *seq* = failure **then** **return** a null action

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*

Tree Search

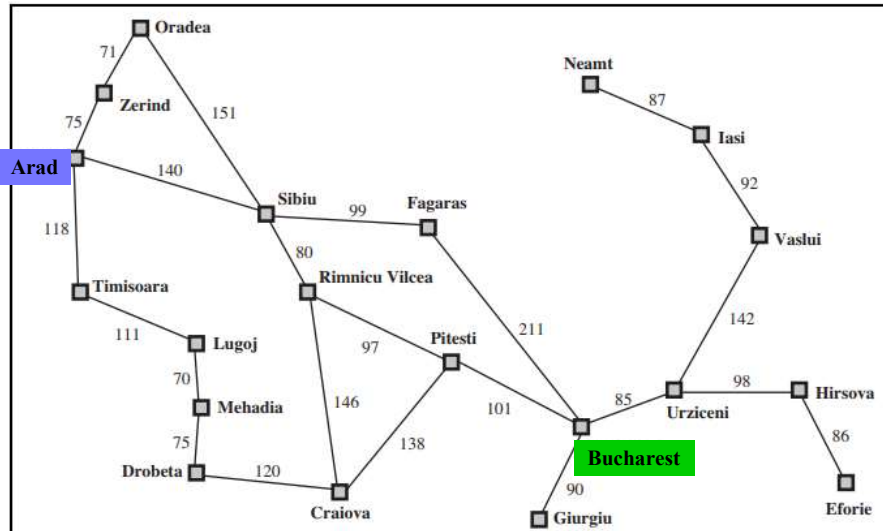
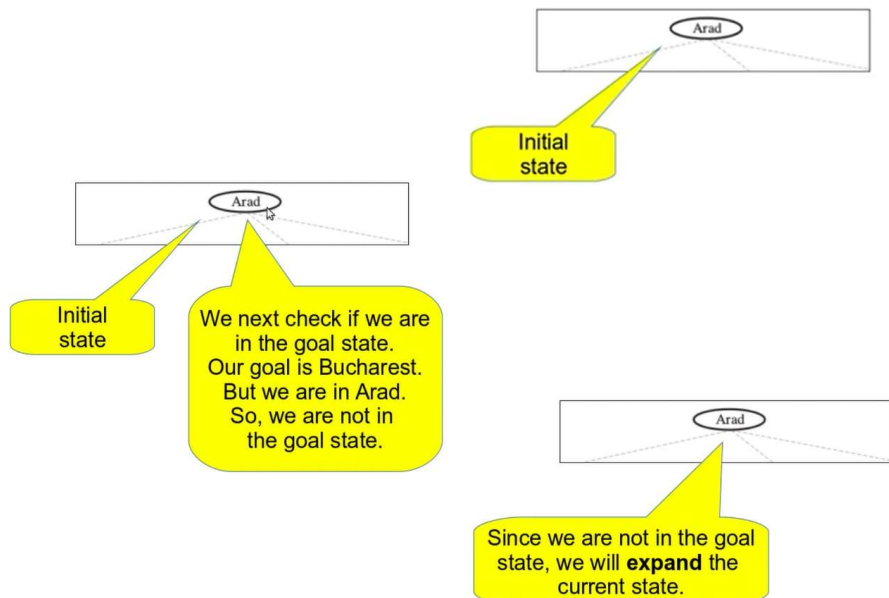
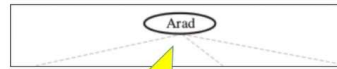


Figure 3.2 A simplified road map of part of Romania.

Tree Search

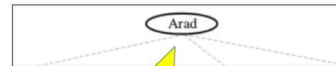


Tree Search



Since we are not in the goal state, we will **expand** the current state.

Expanding the current state means applying each legal action to the current state, thereby **generating** a new set of states

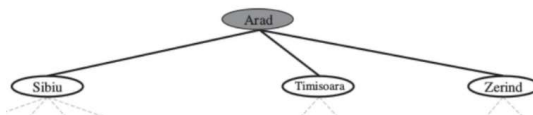


Since we are not in the goal state, we will **expand** the current state.

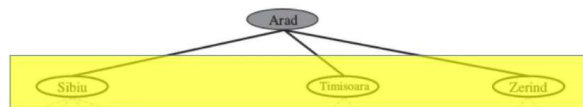
For the current state, In(Arad), the valid actions are Go(Sibiu), Go(Timisoara), and Go(Zerind).

Expanding the current state means applying each legal action to the current state, thereby **generating** a new set of states

Tree Search

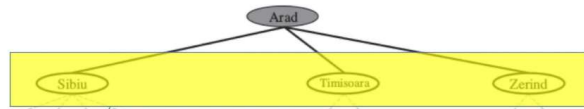


After expanding the current state



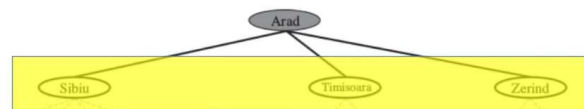
These are the 3 leaf nodes we have now

Tree Search



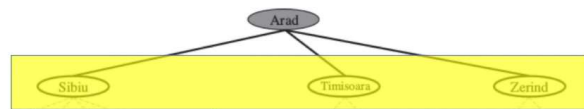
These are the 3 **leaf nodes** we have now

The set of all leaf nodes available for expansion at any given point is called the **frontier** or **open list**.



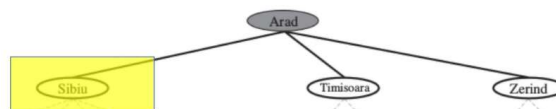
We now need to decide which of these three states to consider next

Tree Search



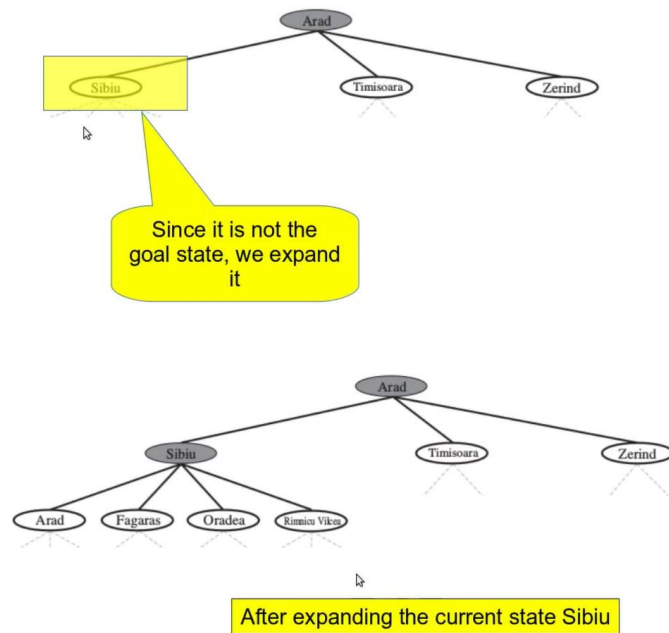
Which node is selected next for expansion depends on the **search strategy**

We now need to decide which of these three states to consider next

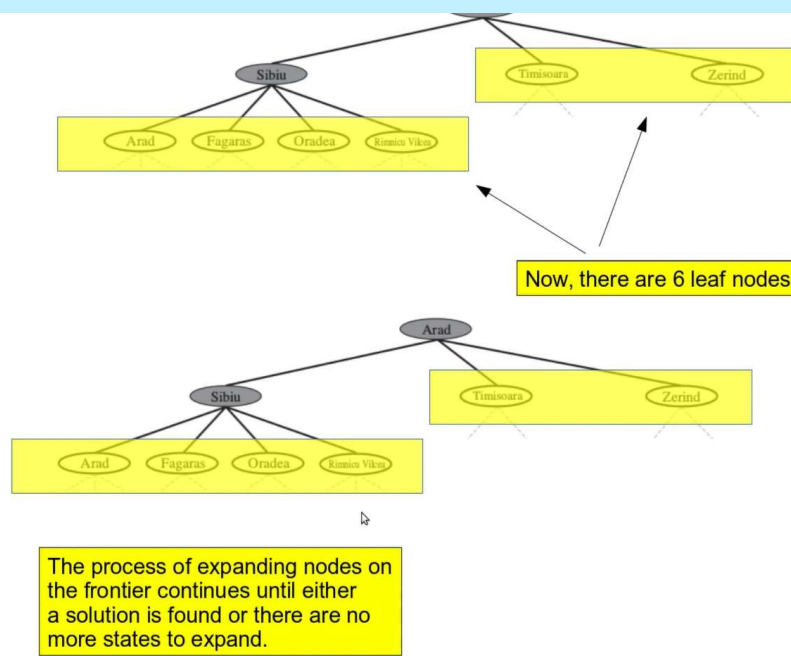


First we check if it is the goal state

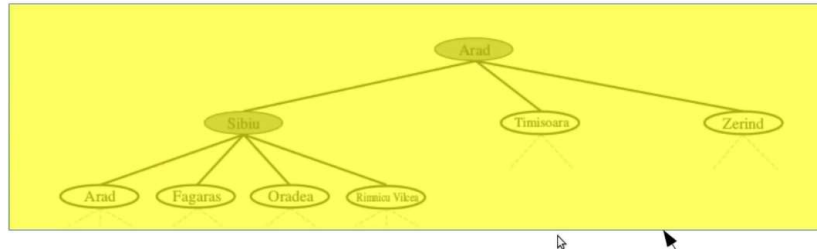
Tree Search



Tree Search



Tree Search



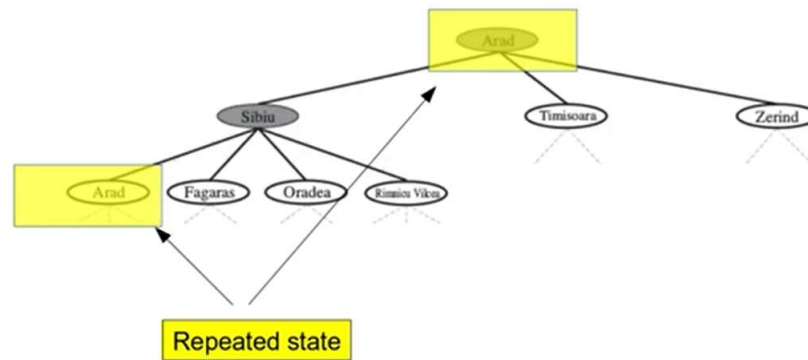
This tree that we are generating is called the **search tree**. Search trees have the initial state at the root; the branches are actions and the nodes correspond to states in the state space of the problem.

Tree Search

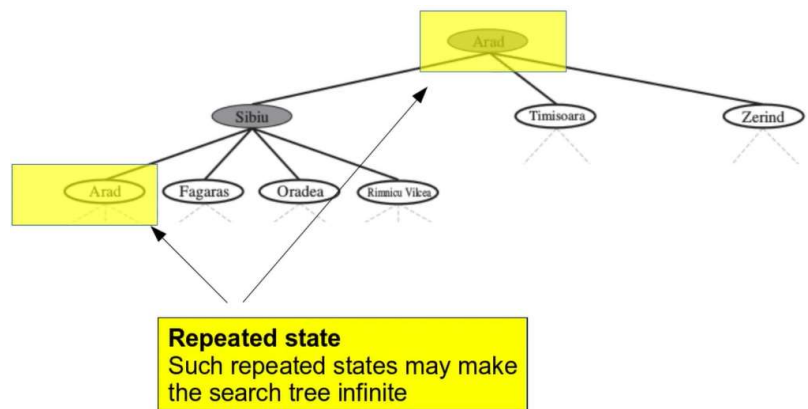
```

function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
  
```

Graph Search



Graph Search



Graph Search

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set

```

Figure 3.7 An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

Activate Windows
Go to Settings to activate Windows.

Graph Search

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set

```

Graph Search

